

HARDWARE ARCHITECTURE AND DESIGN SPECIFICATION

Airbag Electronic Control Unit (ECU)

HADS-AIRBAG-ECU-001 | Revision 2.1 | 2026-06-11

ISO 26262:2018 (ASIL D) / IEC 61508:2010 (SIL 3)

Based on: HARA-AIRBAG-ECU-001 Rev 2.1 | FSRS-AIRBAG-ECU-001 Rev 2.1 | SC-AIRBAG-ECU-001 Rev 2.0 |
SRS-AIRBAG-SW-001 Rev 1.0

REVISION 2.0 — KEY CHANGES FROM REV 1.0

1. Architecture updated: peer-equal (no master / no slave). Both ECUs identical — only ECU_ID, DATA_ID, REMOTE_ID differ.
2. Code reviewed and confirmed: $\text{voteDeploy} = \text{localCrash} \text{ AND } \text{remoteCrash} \text{ AND } \neg \text{systemFault} \text{ AND } \neg \text{remoteFault}$. Software AND is retained; hardware AND gate provides additional physical layer.
3. All LEDs confirmed red (airbag 1, airbag 2, warning lamp). No blue or amber LEDs in this implementation.
4. HADS-CR-001 closed — pull-up moved to 3.3 V. All critical issues resolved.

1. Document Information

Field	Value
Document Number	HADS-AIRBAG-ECU-001
Revision	2.0
Date	2026-06-11
Author	Hardware Safety Engineering Team
Status	DRAFT
ASIL Classification	ASIL D (ISO 26262) / SIL 3 (IEC 61508)
Architecture	Peer-equal dual-channel — NO master / NO slave
Target Hardware	Arduino Due (SAM3X8E) x2, DM74LS08N, WaveShare SN65HVD230 x2, USB-CAN-A
Diagnostic Tool	USBCANv2 software (Python dashboard removed — out of scope)
Code baseline	Confirmed firmware — ECU1 and ECU2 identical except first 3 #define lines

2. Purpose and Scope

This Hardware Architecture and Design Specification (HADS) defines the complete hardware design of the Airbag ECU laboratory prototype. It covers system architecture, bill of materials, wiring connections, pin assignments, electrical connections, power architecture, and functional descriptions of all hardware components.

The document reflects the confirmed firmware architecture where both ECUs operate as peers — there is no master and no slave. Each ECU independently reads sensors, evaluates crash conditions, transmits its own status on its own CAN ID, receives the remote ECU status, and makes its own deployment vote. The hardware AND gate provides a final physical 2oo2 layer on top of the software vote.

Scope: sensor input circuits, processing boards, AND gate deployment logic, output indicator LEDs (all red), CAN bus communication hardware, power distribution, and grounding.

Out of scope: physical airbag inflator, airbag cushion, Python diagnostic dashboard (removed — replaced by USBCANV2).

3. System Architecture Overview

3.1 Peer-equal design principle

Both ECUs are identical in hardware and nearly identical in firmware. Neither ECU is a master or slave. The only differences between ECU 1 and ECU 2 are the three configuration defines at the top of the firmware:

Define	ECU 1 (Due 1)	ECU 2 (Due 2)
ECU_ID	1	2
DATA_ID	0x100	0x200
REMOTE_ID	0x200	0x100

All hardware wiring, all sensor connections, all output connections, and all firmware logic are identical between the two boards.

3.2 Deployment decision architecture

The system implements two independent AND conditions that must both be satisfied before an airbag LED fires:

- Software AND (in firmware, identical on both ECUs): voteDeploy = localCrash AND remoteCrash AND NOT systemFault AND NOT remoteFault. Each ECU independently confirms a crash with its own sensors before asserting its output pins HIGH.
- Hardware AND (DM74LS08N): the physical AND gate ensures the airbag LED physically cannot illuminate unless BOTH ECUs have simultaneously asserted their output pin HIGH. This operates independently of all software.

This two-layer approach means a single software fault or single hardware fault in either channel cannot cause unintended deployment. This satisfies FSR-03 and FSR-08.

3.3 Node inventory

Node	Component	CAN TX ID	CAN RX ID	Role
ECU 1	Arduino Due 1	0x100	0x200	Peer — reads sensors, votes, transmits status
ECU 2	Arduino Due 2	0x200	0x100	Peer — reads sensors, votes, transmits status
OMS lab PC	USBCANV2 software	—	0x100 and 0x200	Passive monitor — logs both channels
CAN transceiver 1	Waveshare SN65HVD230	—	—	ECU 1 CAN0 to CANH/CANL
CAN transceiver 2	Waveshare SN65HVD230	—	—	ECU 2 CAN0 to CANH/CANL
CAN adapter	USB-CAN-A	—	—	CANH/CANL to USB for OMS PC

4. Bill of Materials

Item	Description	Qty	Specification	Safety role	FSR
Arduino Due	SAM3X8E ARM Cortex-M3 84 MHz microcontroller	2	3.3 V logic, 12-bit ADC, CAN0 hardware	Processing channel — ASIL D	FSR-07
Potentiometer 10 kΩ	Rotary potentiometer — sensor simulation	3	Linear taper, 10 kΩ, 3-terminal	Accel / gyro / pressure simulation	FSR-01
Tactile push button	Momentary push button — baby seat switch	1	Active-LOW, SPST, normally open	Baby seat occupant detection — ASIL D	FSR-09
DM74LS08N	Quad 2-input AND gate IC 14-pin DIP	1	74LS TTL, VCC 5 V, VIH min 2.0 V	Hardware 2oo2 physical layer — ASIL D	FSR-03
Waveshare SN65HVD230	CAN transceiver board	2	3.3 V supply, 500 kbit/s, CANH/CANL output	CAN bus physical layer — ASIL B	FSR-13
USB-CAN-A	USB to CAN bus adapter	1	500 kbit/s, USBCANv2 compatible	OMS PC CAN interface	FSR-16
Resistor 10 kΩ	Pull-up for baby seat switch on D11	1	1/4 W — connect to 3.3 V (NOT 5 V)	Safe default HIGH state for D11	FSR-10
Resistor 220 Ω — airbag 1	LED current limiter — airbag 1	1	1/4 W	Current limit for airbag 1 LED	—
Resistor 220 Ω — airbag 2	LED current limiter — airbag 2	1	1/4 W	Current limit for airbag 2 LED	—
Resistor 220 Ω — warning	LED current limiter — warning lamp	2	1/4 W (one per ECU D10 output)	Current limit for DWI LED	FSR-19
LED red — airbag 1	Airbag 1 deployment indicator	1	3 mm RED LED, Vf ~2.0 V	Simulates driver airbag deployment	—
LED red — airbag 2	Airbag 2 deployment indicator	1	3 mm RED LED, Vf ~2.0 V	Simulates passenger airbag deployment	—
LED red — warning lamp	Diagnostic Warning Indicator (DWI)	1	3 mm RED LED, Vf ~2.0 V — shared anode	Warning lamp from both ECU D10 pins	FSR-19
Breadboard	Full-size 830 tie-point solderless breadboard	1	Standard 0.1 inch pitch	All passive and sensor wiring	—
Jumper wires M-M	Dupont jumper wire set	1 set	Various lengths	All breadboard connections	—
Twisted pair wire	CANH/CANL bus wiring	30 cm	Min 1 twist per cm	CAN bus noise immunity	FSR-13

5. Pin Assignments and Electrical Connections

Both ECUs use identical pin assignments. The table below applies to Arduino Due 1 (ECU 1) and Arduino Due 2 (ECU 2) without distinction.

5.1 Arduino Due — pin assignment (identical for both ECU 1 and ECU 2)

Pin	Signal	Connected to	Dir	Notes
A0	Accelerometer POTI-1	Potentiometer 1 wiper (leg 3)	In	0–3.3 V analog, 12-bit ADC, shared with other ECU — high impedance safe
A1	Gyroscope POTI-2	Potentiometer 2 wiper (leg 3)	In	0–3.3 V analog, 12-bit ADC, shared
A2	Pressure POTI-3	Potentiometer 3 wiper (leg 3)	In	0–3.3 V analog, 12-bit ADC, shared
D8	Airbag 1 AND gate input	DM74LS08N pin 1 (ECU1) or pin 2 (ECU2)	Out	HIGH = vote to deploy AB1; LOW = inhibit
D9	Airbag 2 AND gate input	DM74LS08N pin 4 (ECU1) or pin 5 (ECU2)	Out	HIGH = vote to deploy AB2; LOW = inhibit
D10	Warning lamp	220 Ω → DWI LED positive (shared anode)	Out	HIGH = any fault. Both ECUs drive same LED — OR logic
D11	Baby seat switch	Push button + 10 k Ω pull-up to 3.3 V	In	Active-LOW. LOW=baby seat. HIGH=adult/empty. Pull-up connected to 3.3 V — correct.
D13	Status LED	Built-in LED on Arduino Due board	Out	Blinks 500 ms — indicates loop running
CAN TX	CAN transmit	Waveshare CANTX pin	Out	CAN0 controller — ECU1 ID 0x100, ECU2 ID 0x200
CAN RX	CAN receive	Waveshare CANRX pin	In	ECU1 listens for 0x200, ECU2 listens for 0x100
3.3 V	3.3 V supply	Waveshare VCC pin	Pwr	Powers SN65HVD230 — must be 3.3 V not 5 V
5 V	5 V supply	Common breadboard 5 V rail	Pwr	Feeds DM74LS08N VCC pin 14
GND	Ground	Common GND rail, Waveshare GND	Pwr	Single common reference — mandatory

5.2 DM74LS08N AND gate — pin assignment

Pin	Name	Connected to	Function
1	1A	ECU 1 (Due 1) D8	Airbag 1 deploy vote from Channel A
2	1B	ECU 2 (Due 2) D8	Airbag 1 deploy vote from Channel B
3	1Y	220 Ω resistor → Airbag 1 LED positive (anode)	HIGH only when both D8 pins HIGH
4	2A	ECU 1 (Due 1) D9	Airbag 2 deploy vote from Channel A
5	2B	ECU 2 (Due 2) D9	Airbag 2 deploy vote from Channel B
6	2Y	220 Ω resistor → Airbag 2 LED positive (anode)	HIGH only when both D9 pins HIGH
7	GND	Common breadboard GND rail	IC ground reference

Pin	Name	Connected to	Function
8–13	n/c	Unconnected	Gates 3 and 4 unused — leave floating
14	VCC	Common breadboard 5 V rail	IC power — 5 V

5.3 Waveshare SN65HVD230 CAN transceiver (applies to both boards)

Pin	Connected to	Notes
CANTX	Arduino Due CAN TX pin	Logic-level TX input from CAN0 controller
CANRX	Arduino Due CAN RX pin	Logic-level RX output to CAN0 controller
3.3 V	Arduino Due 3.3 V pin	VCC — MUST be 3.3 V. Do NOT connect to 5 V
GND	Common GND rail	Signal reference — common with Arduino Due GND
CANH	Common CANH bus rail	Differential CAN high — shared by both transceivers and USB-CAN-A
CANL	Common CANL bus rail	Differential CAN low — shared by both transceivers and USB-CAN-A

5.4 USB-CAN-A adapter

Pin	Connected to	Notes
CANH	Common CANH bus rail	Connects to both Waveshare CANH pins
CANL	Common CANL bus rail	Connects to both Waveshare CANL pins
USB	OMS lab PC USB port	USBCANv2 software — set to 500 kbit/s

6. Sensor Input Connections

6.1 Potentiometer wiring — confirmed from code

The code uses `analogReadResolution(12)` — all ADC readings are 12-bit (0 to 4095). Both ECUs read identical shared potentiometer signals. The conversion formulas confirmed in firmware are:

```
accel = (rawA0 / 4095.0) * 100.0 // 0-100 g, threshold 30.0 g
gyro  = (rawA1 / 4095.0) * 360.0 // 0-360 deg/s, threshold 120.0 deg/s
press = (rawA2 / 4095.0) * 500.0 // 0-500 kPa, threshold 200.0 kPa
```

Terminal	Connected to	Notes
Leg 1 (end)	Common 3.3 V rail (RECOMMENDED — see HADS-OB-002)	Top of resistive track. Use 3.3 V to constrain wiper to safe ADC range
Leg 2 (end)	Common GND rail	Bottom of resistive track
Leg 3 (wiper)	ADC pin of BOTH Due 1 and Due 2 (shared wire)	Output voltage 0–3.3 V. Both ADC inputs are high-impedance — safe to share

Potentiometer	Due 1 pin	Due 2 pin	Physical range	Deploy threshold	ADC threshold (approx)
POTI-1 Accelerometer	A0	A0	0–100 g	30.0 g	raw > 1228
POTI-2 Gyroscope	A1	A1	0–360 deg/s	120.0 deg/s	raw > 1365
POTI-3 Pressure	A2	A2	0–500 kPa	200.0 kPa	raw > 1638

6.2 Baby seat switch — confirmed from code

The code reads: `babySeat = (digitalRead(PIN_BABY) == LOW)`. Pin D11, configured as INPUT (external pull-up).

Connection	Detail
Switch terminal 1	Common GND rail — pressing button pulls D11 to 0 V
Switch terminal 2	D11 of Due 1 AND D11 of Due 2 (shared) AND 10 kΩ pull-up
Pull-up resistor	10 kΩ — one end to D11 signal line, other end to Arduino Due 3.3 V pin
Logic HIGH (open)	~3.3 V via pull-up — no baby seat / adult occupant
Logic LOW (pressed)	~0 V — baby seat present — passenger airbag suppressed

7. Output Connections

All LEDs in this implementation are red (3 mm, V_f approximately 2.0 V). No blue or amber LEDs are used.

7.1 Airbag 1 LED — driver airbag indicator (RED)

The AND gate output (pin 3) drives the LED directly through a 220 Ω resistor.

Connection	Detail
AND gate pin 3 (1Y)	→ 220 Ω resistor → Airbag 1 LED positive (anode)
LED positive (anode)	→ 220 Ω resistor → Airbag 1 LED positive (anode)
LED cathode	→ Common GND rail
LED cathode	→ Common GND rail
LED colour	RED — $V_f \sim 2.0$ V
LED current	3 mm red LED, $V_f \sim 2.0$ V, direct drive from AND gate output
Active condition	AND gate pin 3 HIGH — requires ECU1 D8 HIGH AND ECU2 D8 HIGH AND voteDeploy TRUE on both

7.2 Airbag 2 LED — passenger airbag indicator (RED)

Same circuit as Airbag 1. AND gate pin 6 drives the LED directly through a 220 Ω resistor.

Connection	Detail
AND gate pin 6 (2Y)	→ 220 Ω resistor → Airbag 2 LED positive (anode)
LED positive (anode)	→ 220 Ω resistor → Airbag 2 LED positive (anode)
LED cathode	→ Common GND rail
LED colour	RED — 3 mm red LED, same type as Airbag 1
Active condition	AND gate pin 6 HIGH — requires ECU1 D9 HIGH AND ECU2 D9 HIGH. D9 is only HIGH when voteDeploy=TRUE AND babySeat=FALSE

7.3 Warning lamp — DWI (RED LED, shared between both ECUs)

Both ECU D10 pins connect to the same LED through separate 220 Ω resistors. Either pin driving HIGH illuminates the lamp — OR gate behaviour implemented in hardware.

Connection	Detail
ECU 1 D10	→ 220 Ω resistor → DWI LED positive (anode)
ECU 2 D10	→ 220 Ω resistor → same DWI LED positive (anode)
LED cathode	→ Common GND rail
LED colour	RED — 3 mm red LED, same type as airbag LEDs
Active condition	Either ECU detects systemFault = TRUE (CPU / RAM / ADC / COMM fault)
FSR	FSR-19 — DWI must activate within 800 ms of fault detection

8. CAN Bus Architecture

8.1 Physical bus configuration

Parameter	Value
Bus topology	Single segment — three nodes on one twisted pair
Baud rate	500 kbit/s — confirmed in firmware: <code>Can0.begin(CAN_BPS_500K)</code>
Termination	120 Ω at each physical end of the bus — two resistors total
Cable type	Twisted pair — minimum 1 twist per cm
Bus nodes	Waveshare 1 (ECU 1), Waveshare 2 (ECU 2), USB-CAN-A (OMS PC)
Terminator position A	At the ECU 1 / Waveshare 1 end of the bus
Terminator position B	At the USB-CAN-A / OMS PC end of the bus
Waveshare 2 position	Middle node — no terminator on Waveshare 2

8.2 CAN ID assignment — confirmed from firmware

Board	TX ID	RX filter	Frame rate	Receive filter code
Arduino Due 1 (ECU 1)	0x100	Accept all (watchFor with no filter)	~500 ms (heartbeat loop)	<code>Can0.watchFor()</code> — accepts all IDs. ECU filters by checking <code>incoming.id == REMOTE_ID (0x200)</code> in software
Arduino Due 2 (ECU 2)	0x200	Accept all (watchFor with no filter)	~500 ms	Same logic — filters for <code>incoming.id == 0x100</code> in <code>receiveCAN()</code>
OMS PC USBCANV2	—	0x100 and 0x200	Passive	Configure USBCANV2 to display both IDs

8.3 CAN frame byte layout — confirmed from `sendData()` function

Byte	Field	Encoding	Code reference
0	A0 raw low byte	<code>rawA0 & 0xFF</code>	<code>frame.data.bytes[0] = a0 & 0xFF</code>
1	A0 raw high byte	<code>rawA0 >> 8</code>	<code>frame.data.bytes[1] = a0 >> 8</code>
2	A1 raw low byte	<code>rawA1 & 0xFF</code>	<code>frame.data.bytes[2] = a1 & 0xFF</code>
3	A1 raw high byte	<code>rawA1 >> 8</code>	<code>frame.data.bytes[3] = a1 >> 8</code>
4	A2 raw low byte	<code>rawA2 & 0xFF</code>	<code>frame.data.bytes[4] = a2 & 0xFF</code>
5	A2 raw high byte	<code>rawA2 >> 8</code>	<code>frame.data.bytes[5] = a2 >> 8</code>
6	Status byte	Bit0=crash, Bit1=babySeat, Bit2=fault	<code>frame.data.bytes[6] = status (bit flags)</code>
7	ECU ID	1 or 2	<code>frame.data.bytes[7] = ECU_ID</code>

Note: This frame format transmits raw ADC values (12-bit, split across two bytes). The OMS PC (USBCANV2) must reconstruct the 16-bit value as: `raw = (byte1 << 8) | byte0`, then convert to physical units using the same formula as the firmware.

8.4 Status byte bit fields (byte 6)

Bit	Field	Value	Meaning
Bit 0	localCrash	1 = crash detected	At least one sensor exceeded threshold on this ECU
Bit 1	babySeat	1 = baby seat present	D11 LOW — passenger airbag suppressed on this ECU
Bit 2	systemFault	1 = fault active	CPU, RAM, ADC, or COMM fault on this ECU
Bit 3–7	Reserved	0	Unused in current firmware

9. Deployment Logic — Confirmed from Firmware

9.1 Software vote — identical on both ECUs

The vote decision confirmed from firmware (same logic on both ECUs):

```
localCrash  = (accel > 30.0) || (gyro > 120.0) || (press > 200.0)
systemFault = faultCPU || faultRAM || faultADC || faultCOMM

voteDeploy  = localCrash && remoteCrash && !systemFault && !remoteFault

D8 (Airbag 1 AND gate input) = voteDeploy
D9 (Airbag 2 AND gate input) = voteDeploy && !babySeat
D10 (Warning lamp)           = systemFault
```

remoteCrash and remoteFault are extracted from the received CAN frame byte 6 of the other ECU:

```
remoteCrash = (status & 0x01) // bit 0 of received byte 6
remoteFault = (status & 0x04) // bit 2 of received byte 6
```

9.2 Communication timeout fault

The firmware monitors the remote CAN frame reception timestamp:

```
if (millis() - lastRemoteHB > 2000) faultCOMM = true
```

If no frame is received from the remote ECU for 2000 ms, faultCOMM becomes TRUE. systemFault becomes TRUE. voteDeploy becomes FALSE. D8 and D9 are driven LOW. The AND gate output drops LOW. No airbag LED fires. This is the safe state on communication loss.

9.3 Hardware AND gate — final physical layer

On top of the software vote, the DM74LS08N AND gate provides a physical 2oo2 layer:

ECU1 D8	ECU2 D8	AND gate pin 3 output	Airbag 1 LED
LOW	LOW	LOW	OFF — no crash on either
HIGH	LOW	LOW	OFF — ECU2 not confirming
LOW	HIGH	LOW	OFF — ECU1 not confirming
HIGH	HIGH	HIGH	FIRE — both ECUs confirm crash

10. Power Architecture

10.1 Power rails

Rail	Source	Voltage	Consumers	Notes
5 V	Arduino Due 5 V pin (USB powered)	5 V	DM74LS08N VCC (pin 14) only	Shared rail on breadboard. Baby seat pull-up uses 3.3 V pin, not this rail.
3.3 V — ECU 1	Arduino Due 1 3.3 V pin	3.3 V	Waveshare 1 VCC, 10 k Ω baby seat pull-up	Powers CAN transceiver 1 and baby seat 10 k Ω pull-up resistor.
3.3 V — ECU 2	Arduino Due 2 3.3 V pin	3.3 V	Waveshare 2 VCC	Powers CAN transceiver 2
GND	Common breadboard GND rail	0 V	All components	Single ground reference — mandatory for all signal integrity

10.2 Voltage domain boundaries

Interface	Concern	Status	Action required
Potentiometer leg 1 to ADC	If leg 1 = 5 V, wiper can reach 5 V — over-voltages 3.3 V ADC	OPEN — HADS-OB-002	Move leg 1 from 5 V to 3.3 V rail
Baby seat pull-up to D11	Pull-up connected to 3.3 V — D11 reads 3.3 V when open. No over-voltage.	RESOLVED — HADS-CR-001 closed	Hardware updated — pull-up now on 3.3 V. No further action required.
Due D8/D9 (3.3 V) to DM74LS08N input	74LS VIH min = 2.0 V — 3.3 V is sufficient	OK — no action	Verified safe
DM74LS08N output to LED (direct drive)	74LS VOH ~2.7 V — 3 mm red LED with Vf ~2.0 V, direct drive confirmed working	OK — confirmed	Direct drive — 3 mm LED confirmed in use
Waveshare VCC	SN65HVD230 requires 3.3 V — 5 V will destroy IC	Must verify wiring	Confirm 3.3 V connected, not 5 V

11. Functional Safety Engineer Review — Open Issues

Issues marked **CRITICAL** must be corrected before any V&V testing.

HADS-CR-001 — CLOSED: Pull-up moved to 3.3 V — resolved

Previous issue: 10 k Ω baby seat pull-up was connected to 5 V rail. D11 was reading 5 V when button open. Risk was: Arduino Due GPIO maximum input is 3.3 V. A 5 V signal on D11 exceeded the absolute maximum rating of the SAM3X8E.

Resolution applied: The free end of the 10 k Ω pull-up resistor has been moved from the 5 V breadboard rail to the Arduino Due 3.3 V pin. D11 now reads 3.3 V when button open (valid) and 0 V when pressed (valid). No firmware change required.

FSR: FSR-09, FSR-10, FSR-11 — ASIL D | Status: CLOSED — pull-up moved to 3.3 V

HADS-OK-002 — Confirmed correct: direct LED drive working (3 mm red LEDs)

Implementation: DM74LS08N output pins 3 and 6 connected directly to 3 mm red airbag LEDs through 220 Ω resistors.

Status: Direct drive confirmed working in the laboratory. 3 mm red LEDs operate correctly with this circuit. No issues observed.

Implementation: AND gate output $\rightarrow$ 220 Ω $\rightarrow$ 3 mm red LED anode $\rightarrow$ LED cathode $\rightarrow$ GND. Confirmed working in the laboratory.

FSR: FSR-03, FSR-08, FSR-20 | Status: OPEN

HADS-OB-001 — WARNING: Waveshare board onboard termination — verify

Some Waveshare SN65HVD230 boards have an onboard 120 Ω termination resistor. If enabled on both boards, the CAN bus will have too many terminators causing communication errors.

Action: Inspect each Waveshare board for a termination jumper or onboard resistor near CANH/CANL. The bus needs exactly two 120 Ω terminators — one at each physical end. Disable onboard termination if present. | Status: OPEN — verify before CAN commissioning

HADS-OB-002 — ADVISORY: Potentiometer leg 1 connected to 5 V

Potentiometer leg 1 connected to 5 V rail. At full clockwise rotation wiper approaches 5 V, over-volting the 3.3 V ADC input.

Recommended fix: move potentiometer leg 1 from 5 V to 3.3 V rail. Wiper then ranges 0–3.3 V — exactly the ADC input range. Full 0–4095 range preserved. | Status: ADVISORY — recommend fixing

Code observation HADS-CODE-OB-001 — NOTE: watchdog not implemented in current firmware

The current firmware does not include the hardware watchdog (WDT) implementation from the earlier safety concept. FSR-22 requires hardware watchdog supervision. The `sam.h` `watchdog_setup()` and `watchdog_feed()` functions should be added to the firmware.

Status: ADVISORY — add before final V&V

Confirmed correct — Peer-equal architecture

Both ECUs are independent peers. No master/slave dependency. Each ECU independently reads sensors, votes, and transmits. If one ECU fails, its output goes LOW, the AND gate output goes LOW, no deployment occurs. Communication timeout correctly handled by `faultCOMM`. Architecture is correct for ASIL D.

Confirmed correct — CAN frame reception

`receiveCAN()` is called at the top of every loop iteration. `Can0.watchFor()` accepts all IDs. Software filter (`incoming.id == REMOTE_ID`) correctly isolates peer frames. `lastRemoteHB` is updated on every valid peer frame. `faultCOMM` activates after 2000 ms silence. Correct.

Confirmed correct — All LEDs red

All three LEDs (airbag 1, airbag 2, warning lamp) are 3 mm red LEDs with V_f approximately 2.0 V. No blue or amber LEDs.

12. Hardware Items to Replace or Modify

Item	Current	Change to	Priority	Reason
Baby seat pull-up — supply connection	5 V rail	3.3 V (Arduino Due 3.3 V pin)	CONFIRMED — no change	HADS-CR-001 closed — MCU input now within 3.3 V safe range
Potentiometer leg 1 — supply connection	5 V rail	3.3 V rail	RECOMMENDED	Prevents ADC over-voltage at full rotation (HADS-OB-002)
Airbag LED 1 circuit	AND gate pin 3 → 220 Ω → 3 mm red LED	No change — confirmed correct	CONFIRMED — no change	Confirmed working — 3 mm red LEDs operate correctly with direct drive
Airbag LED 2 circuit	AND gate pin 6 → 220 Ω → 3 mm red LED	No change — confirmed correct	CONFIRMED — no change	Same as above
Airbag LEDs colour	Previously blue/red/amber mixed	All 3 mm red LEDs	DONE — confirmed	All three LEDs are red
Waveshare onboard termination	Unknown state	Verify and disable if present	IMPORTANT	Excess termination causes CAN errors (HADS-OB-001)
Hardware watchdog in firmware	Not implemented	Add sam.h WDT setup and feed calls	RECOMMENDED	Required by FSR-22 for ASIL D compliance (HADS-CODE-OB-001)
Python dashboard	Removed in Rev 1.0	USBCANv2 confirmed	DONE	Out of project scope

13. Revision History

Rev	Date	Author	Change
1.0	2026-06-11	Hardware Safety Engineering Team	Initial release. Full HADS covering confirmed breadboard wiring, BOM, pin assignments, power architecture, CAN bus, safety review with two critical issues. Python dashboard removed — USBCANv2 confirmed.
2.0	2026-06-11	Hardware Safety Engineering Team	Updated to peer-equal architecture (no master/slave). Confirmed firmware reviewed — voteDeploy logic, CAN frame format, byte layout, and timeout handling documented. All LEDs confirmed as 3 mm red. Direct LED drive from AND gate outputs confirmed correct. Watchdog observation added. HADS-CR-001 closed — baby seat pull-up moved from 5 V to 3.3 V. No critical issues remain open.